

2026-06-10-F-parallel-dynamic-testing

Workstream F — Parallel/Subagent Enforcement · Dynamic-Flow Submodules · Testing/QA Strategy

Phase: PLANNING (read-only analysis). **Date:** 2026-06-10. **Author:** Workstream-F analysis subagent. **Scope:** helix_code/ (inner Go app dev.helix.code), submodules/helix_agent/ (HelixAgent), submodules/helix_qa/, challenges/, plus own-org orchestration submodules already wired in .gitmodules. **Evidence rule:** every claim cites a real file:line or dir/. ABSENT is marked explicitly. No fabrication.

1. Parallel / subagent-driven coordination — WHAT EXISTS

1.1 HelixCode (helix_code/internal/...)

Capability	Location	Evidence
Multi-agent coordinator (registry, task queue, results, circuit breakers, retry, conflict resolution by voting)	helix_code/internal/agent/coordinator.go:11-50	type Coordinator struct{ registry, tasks, taskQueue, results, workflowExecutor, circuitBreakers, retryPolicy }; CoordinatorConfig.MaxConcurrentTasks default 10, ConflictResolution: ResolutionMethodVoting (coordinator.go:38-49)
Multi-step workflow over agents with DAG deps	helix_code/internal/agent/workflow.go:12-46	WorkflowStep.DependsOn []string, Optional bool; Workflow.Steps, Results map[stepID]*task.Result
Agent types (planner/builder/tester/reviewer/debugger/documentor) + Orchestrator	helix_code/internal/agent/README.md	TypePlanner...TypeDocumentor; type Orchestrator struct{ agents, coordinator, config }
Subagent spawning — in-process AND subprocess,	helix_code/internal/agent/subagent/	inprocess_spawner.go, subprocess_spawner.go, manager.go, worktree_integration.go, helper_mode.go

Capability	Location	Evidence
with git-worktree isolation		
Distributed worker pool over SSH (auto-install, heartbeats, capability-based assignment, isolation, consensus)	helix_code/internal/worker/	worker_pool.go, ssh_pool.go, ssh_pool_consensus.go, consensus.go, distributed_manager.go, isolation.go; README lists “Consensus management for distributed coordination”
Priority task queue + dependency resolution + checkpointing	helix_code/internal/task/	queue.go (highPriority/normalPriority/lowPriority in GetNextTask), dependency.go, checkpoint.go, manager.go
Standalone DAG workflow executor (step deps, LLM-powered steps)	helix_code/internal/workflow/	workflow.go:12-26 (race-guarded Workflow), executor.go, README “Step dependencies (DAG execution)”
Autonomy controller (modes, guardrails, escalation, permission)	helix_code/internal/workflow/autonomy/	controller.go, guardrails.go, escalation.go, permission.go, modes.go
Plan-mode gating + planner approval	helix_code/internal/workflow/planmode/	mode_controller.go, gating.go, planner_approval_test.go
Background workflow execution	helix_code/internal/workflow/background.go	background_test.go present
Planner / plan-tree / continuation tooling	helix_code/internal/planner/, .../ plantree/, .../ continua/	planner/executor.go, plantree/operations.go+store.go, continua/session.go

Verdict (HelixCode): Substantial parallel-agent machinery EXISTS and is self-contained inside helix_code/internal/ (own coordinator + subagent spawner + SSH worker pool + DAG workflow + autonomy). It is NOT consumed from the standalone own-org orchestration submodules.

1.2 HelixAgent (submodules/helix_agent/internal/...)

Capability	Location	Evidence
Graph-based (LangGraph-style) agentic workflow engine	submodules/ helix_agent/ internal/agentic/ workflow.go:28-194	WorkflowGraph, Node, NodeType, Edge, NodeHandler, WorkflowState, NodeExecution; README: node types agent/tool/condition/parallel/human
Agent registry (18+ CLI agents) + sub-namespaces dream/kairos/modes/subagent/swarm/voice/yolo	submodules/ helix_agent/ internal/agents/	registry.go; dirs dream/ kairos/ modes/ subagent/ swarm/ voice/ yolo/
Subagent orchestrator + manager	submodules/ helix_agent/ internal/agents/ subagent/	orchestrator.go, manager.go, types.go
Multi-agent swarm (XML comms, colors, roles — Claude-Code-team-style)	submodules/ helix_agent/ internal/agents/ swarm/ swarm.go:1-30	package swarm “multi-agent swarm enhancements”, AgentColor, AgentRole; imports digital.vasic.concurrency/pkg/safe
Concurrency primitives (semaphore, non-blocking, worker pool, deadlock detection)	submodules/ helix_agent/ internal/ concurrency/	semaphore.go, nonblocking.go, worker_pool.go, deadlock/
Advanced planning: Tree-of-Thoughts, MCTS (MASTER framework), Hierarchical	submodules/ helix_agent/ internal/planning/	tree_of_thoughts.go:1, mcts.go:1-13 (imports digital.vasic.concurrency/pkg/safe), hiplan.go
Background task engine (adaptive worker pool, stuck detector, resource monitor)	submodules/ helix_agent/ internal/ background/	adaptive_worker_pool_test.go, stuck_detector.go, resource_monitor.go, task_queue.go
Ensemble (multi-instance,	submodules/ helix_agent/	

Capability	Location	Evidence
synchronization, background) + placement (capability prober/planner)	internal/ ensemble/,.../ placement/	ensemble/multi_instance/ placement/ capability.go+planner.go+prober.go
CLI instance manager + event bus (drives external CLI agents)	submodules/ helix_agent/ internal/clis/	instance_manager.go, event_bus.go, per-agent dirs aider/ claude/ claude_code/ kiro/ ...

1.3 Own-org orchestration submodules ALREADY in .gitmodules

.gitmodules already wires reusable, decoupled own-org modules (consumed today mainly by HelixAgent):

- submodules/agentica → vasic-digital/Agentica (module digital.vasic.agentica)
- submodules/concurrency → vasic-digital/Concurrency (module digital.vasic.concurrency, go 1.25.0)
- submodules/planning → digital.vasic.planning (go 1.24)
- submodules/llm_orchestrator → HelixDevelopment/LLMOrchestrator
- submodules/debate_orchestrator → HelixDevelopment/DebateOrchestrator

helix_code/go.mod requires own-org digital.vasic.{debate, helixqa, helixspecifier, lazy, containers, challenges} with replace ... => ../submodules/.... It does **NOT** require digital.vasic.{agentica, concurrency, planning} nor LLMOrchestrator. Grep for those imports across helix_code/internal + helix_code/cmd returns ONLY dev.helix.code/internal/agent/subagent (its own internal package).

1.4 THE GAP (parallel/subagent enforcement)

The request: “*all work HelixCode performs can be done by multiple coordinated parallel agents; subagent-driven used as much as possible.*”

1. **Duplication, not reuse (CONST-051/§11.4.74 risk).** HelixCode re-implements coordination/concurrency/planning inside helix_code/internal/{agent, worker, workflow, task} while equivalent decoupled own-org modules (agentica, concurrency, planning, swarm in helix_agent) already exist and are NOT imported. There is no shared coordination contract between HelixCode and HelixAgent — two parallel implementations of the same concern.
2. **No universal “every operation is an agent-dispatchable unit” abstraction.** HelixCode’s coordinator/workflow cover *agent-typed* steps, but ordinary HelixCode operations (model-access, repomap build, LLM generate, command exec) are not uniformly modeled as parallel-dispatchable task units behind one scheduler. Subagent spawning (agent/subagent/) exists but is not the default execution substrate for all work.

3. **Swarm/ensemble parity missing in HelixCode.** HelixAgent has `agents/swarm + ensemble/multi_instance`; HelixCode has voting conflict-resolution in the coordinator but no swarm/ensemble equivalent.
4. **No cross-process agent bus shared with HelixAgent.** HelixAgent has `clis/event_bus.go + instance_manager.go` to drive external agents; HelixCode has no symmetric bus to be driven-by / drive HelixAgent.

Closing the gap = extract the shared coordination/flow concern into decoupled own-org submodule(s) (below), wire BOTH HelixCode and HelixAgent to consume them via `replace/SDK`, and make subagent-dispatch the default path per §11.4.70/§11.4.103.

2. Other execution models / dynamic flows — submodule proposal

2.1 Execution models that EXIST today

Model	Where	Note
DAG / dependency workflow	<code>helix_code/internal/workflow/ (executor.go), helix_code/internal/agent/workflow.go (DependsOn)</code>	static step graph; deps resolved at run
Graph / state-machine (LangGraph-style)	<code>submodules/helix_agent/internal/agent/workflow.go (WorkflowGraph/Node/Edge, node types incl. condition, parallel, human)</code>	dynamic-ish (conditional edges) but lives ONLY in <code>helix_agent</code>
Tree-of-Thoughts / MCTS / Hierarchical planning	<code>submodules/helix_agent/internal/planning/</code>	search-based planning; HelixCode-side ABSENT
Priority queue scheduling	<code>helix_code/internal/task/queue.go</code>	high/normal/low
Autonomy / plan-mode controllers	<code>helix_code/internal/workflow/{autonomy,planmode}/</code>	gating + guardrails
Debate / multi-orchestrator	<code>submodules/{debate_orchestrator,llm_orchestrator}</code>	own-org submodules; HelixCode requires <code>digital.vasic.debate</code>

ABSENT in BOTH repos as named packages: `pipeline`, `dag` (as a standalone reusable engine), `dataflow`, `event-driven/reactive flow`, `saga`, `state-machine` (standalone), `behavior-tree`.
`ls helix_code/internal | grep -iE 'flow|graph|dag|pipeline|orchestr|swarm' → only workflow.`

2.2 Proposed NEW decoupled submodules (under HelixDevelopment, public, created via gh/ghlab)

Per §11.4.74 (catalogue-first) the existing `vasic-digital/Agentic` graph engine + Concurrency + Planning should be the FIRST reuse candidates — **extend them** rather than greenfield where $\geq 80\%$ matches. Net-new flow engines that have no existing home become new submodules. Candidate names (project-not-aware, reusable, modular — CONST-051(B)):

Proposed submodule	Org/repo (to create)	Go module id	Execution model it owns
flow_engine	HelixDevelopment/ FlowEngine	<code>dev.helix.flow</code>	Dynamic flow runtime: typed nodes, conditional dynamic edges, parallel/in/out, human-in-loop, sub-flow nesting, runtime re-planning
dag_orchestrator	HelixDevelopment/ DagOrchestrator	<code>dev.helix.dag</code>	Pure data/dependency DAG scheduler (topological, fan-in/out, partial-failure, retries) decoupled from agents
pipeline_runtime	HelixDevelopment/ PipelineRuntime	<code>dev.helix.pipeline</code>	Linear/branching staged pipeline (transform→validate→serialize) with backpressure
agent_mesh (or swarm_kit)	HelixDevelopment/ AgentMesh	<code>dev.helix.mesh</code>	Multi-agent swarm/ensemble: roles, colored agents, XML/structured comms, voting/consensus, parallel coordination
planning_search	reuse <code>vasic-digital/Planning</code>	<code>digital.vasic.planning</code>	ToT / MCTS / hierarchical search-based planning
flow_dsl (optional)	HelixDevelopment/ FlowDSL	<code>dev.helix.flowdsl</code>	Declarative YAML/JSON flow definition + validation + loader feeding the above engines

Naming respects CONST-052 lowercase snake_case at the directory/path layer (submodules/`flow_engine/`), repo display names CamelCase per host convention.

2.3 Wiring / integration points (interfaces to implement)

Each new engine must expose a small, project-not-aware Go interface; BOTH HelixCode and HelixAgent implement adapters (config-injection only, no hardcoded parent reach — CONST-051(B)).

- **Node/Step contract:** type `Node` interface { `ID()` string; `Run(ctx, *State, Input) (Output, error)`; `DependsOn()` []string } — HelixCode adapts `helix_code/internal/workflow.Step` + `agent.WorkflowStep`; HelixAgent adapts `agentic.Node`.
- **Scheduler contract:** type `Scheduler` interface { `Submit(Unit) Handle`; `Await(Handle) (Result, error)`; `Parallelism()` int } — backed by HelixCode `agent/subagent` spawner and worker SSH pool; backed by HelixAgent `concurrency.WorkerPool` + background.
- **Dispatch sink:** type `Dispatcher` interface { `Dispatch(ctx, Task) (Result, error)` } so a flow node can dispatch to (a) in-process subagent, (b) subprocess subagent, (c) SSH worker, (d) external CLI agent via `helix_agent_cli/event_bus`.
- **Conflict/consensus port:** reuse HelixCode agent voting (`ResolutionMethodVoting`) + `worker/consensus.go`; expose as `Resolver` interface.
- **HelixCode wire-in:** `helix_code/go.mod` add `require dev.helix.flow ...` + replace `dev.helix.flow => ../submodules/flow_engine`; register flow tools in `helix_code/internal/agent/coordinator.go` + `workflow/executor.go` as the execution substrate.
- **HelixAgent wire-in:** replace `internal/agentic` internals with the shared `flow_engine`, keeping `agentic` as a thin adapter; lift `agents/swarm+ensemble` into `agent_mesh`.
- **HelixQA wire-in:** new flows become testable units — register flow banks in `helix_qa/banks/` and challenges in `challenges/`.
- **Manifest:** each new submodule ships `helix-deps.yaml` (CONST-054) declaring concurrency/planning deps so consumers reconstruct the graph at root (no nested own-org chains — CONST-051(C)).

2.4 Deep-web-research topics needed (DO NOT research now — list only)

1. LangGraph / LangChain graph-execution semantics (conditional edges, checkpointing, human-in-loop) — latest API (§11.4.99).
2. Temporal / Cadence durable-workflow + saga patterns in Go (durable execution, signals, queries).
3. Apache Airflow / Dagster / Prefect DAG-scheduler design (dynamic task mapping, partial-retry, backfill).
4. Behavior trees vs state machines vs DAGs for agent control flow — tradeoffs.
5. Actor model / supervision trees (Akka, Proto.Actor, Go *ergo*) for swarm supervision + fault isolation.
6. MCTS / Tree-of-Thoughts / Graph-of-Thoughts / MASTER framework latest papers (validate `planning/mcts.go` claims).
7. Multi-agent orchestration frameworks: AutoGen, CrewAI, OpenAI Swarm, MetaGPT — coordination + handoff protocols.
8. Go concurrency-pattern correctness: structured concurrency, `errgroup`, semaphore-weighted, deadlock detection (validate `concurrency/deadlock`).
9. Workflow-as-data DSL design (YAML/CUE/HCL) + schema validation for `flow_dsl`.

10. Distributed consensus for agent voting (Raft/Paxos-lite) — relate to worker / `ssh_pool_consensus.go`.
 11. Backpressure / flow-control for streaming pipelines (reactive streams) for `pipeline_runtime`.
 12. gh/ghlab repo-creation + branch-protection + multi-upstream (GitHub+GitLab+GitFlic+GitVerse) automation for new public submodules.
-

3. Testing / QA — coverage matrix

3.1 HelixQA (submodules/helix_qa/) — present, must be pulled to latest

- **State:** README banner says **round 219, 2026-05-19** → STALE vs today 2026-06-10.
TASK (do not run now): `git -C submodules/helix_qa fetch --all --prune && git -C submodules/helix_qa pull` then bump pointer per §11.4.71 (do NOT fetch in this PLANNING pass).
- **Structure:** `banks/` (100+ YAML/JSON test banks incl. `helixcode-*.yaml`, `nexus-*`, `ddos-ratelimit-comprehensive.yaml`, `security-comprehensive.yaml`, `benchmarking-baselines.yaml`), `tests/` (`{benchmark,e2e,integration,security,stress}`, `cmd/helixqa` (+ autonomous subcommand), `challenges/scripts/`, `helixqa-bridge`.
- **HelixQA's own ledger** (`submodules/helix_qa/docs/test-coverage.md`) declares all **15 test-type slots FILLED** (unit, integration, e2e, full-automation, security, ddos, scaling, chaos, stress, performance, benchmarking, ui, ux, Challenges, autonomous-QA-session) with per-row evidence pathways.
- **HelixCode[?]HelixQA bridge:** `helix_code/internal/helixqa/` (`{wrapper.go,translator.go}`; Makefile `helixqa-build/helixqa-test/helixqa-challenge/helixqa-bump-submodules` (`helix_code/Makefile:694-720`). `helix_code/go.mod` requires `digital.vasic.helixqa` (replace → `../submodules/helix_qa`).

3.2 HelixCode test-type matrix — WHERE EACH LIVES TODAY

#	Test type	HelixCode location	Status / evidence
1	unit	<code>helix_code/internal/**/*_test.go</code> , <code>helix_code/tests/unit/</code>	PRESENT; Makefile test (Makefile:107), <code>test-unit-full</code>
2	integration	<code>helix_code/tests/integration/</code> (<code>api_integration_test.go</code> , <code>approval_test.go</code> , ...)	PRESENT; <code>test-integration-full (:222)</code> , <code>-tags=integration</code>
3	e2e	<code>helix_code/tests/e2e/</code> (<code>complete_workflow_test.go</code> , <code>challenges/</code> , <code>core/</code> , <code>docker-compose</code>)	PRESENT; <code>test-e2e-full (:228)</code>
4	full-automation	HelixQA autonomous sessions + <code>helix_code/tests/automation/</code>	PRESENT (delegated to HelixQA autonomous); <code>test-complete (:246)</code>
5	security		

#	Test type	HelixCode location	Status / evidence
		helix_code/tests/security/ (owasp_test.go, authn/authz, tools_security_test.go)+make security-scan*(Makefile:751-775: gosec/trivy/grype/kics/semgrep/snyk/sonarqube)	PRESENT; test- security-full (:234)
6	ddos	HelixQA banks/ddos-ratelimit- comprehensive.yaml+challenges/ scripts/ ddos_health_flood_challenge.sh	HelixCode-local harness ABSENT — relies on HelixQA/challenges
7	scaling	HelixQA challenges/scripts/ scaling_horizontal_challenge.sh	HelixCode-local ABSENT — delegated
8	chaos	helix_code/tests/stresschaos/ chaos.go; Makefile stress-chaos (:114) covers ~30 internal pkgs - race	PRESENT (rich)
9	stress	helix_code/tests/stresschaos/ stresschaos.go; per-pkg *_stress_test.go (e.g. internal/worker/ worker_pool_stress_test.go, task/ task_stress_test.go, agent/ agent_stress_test.go, workflow/ workflow_stress_test.go)	PRESENT (rich)
10	performance	helix_code/tests/performance/ (benchmark_test.go, competitor_baseline_test.go, pgo_test.go, pprof_harness_test.go, scenarios/)	PRESENT; test- benchmark (:157)
11	benchmarking	same as 10 + PGO (Makefile:70-95 pgo- refresh)+ HelixQA banks/benchmarking- baselines.yaml	PRESENT
12	ui	applications terminal-ui/desktop tests + HelixQA ui_terminal_interaction_challenge.sh	HelixCode-local UI test harness THIN — mostly delegated to HelixQA
13	ux	HelixQA ux_end_to_end_flow_challenge.sh	HelixCode-local ABSENT — delegated
14	Challenges	challenges/ submodule (banks p1-f06...p2- f24), helix_code/tests/e2e/ challenges/	PRESENT
15	autonomous- QA-session	HelixQA cmd/helixqa autonomous+pkg/ {autonomous, issuedetector, session}	PRESENT (in HelixQA)
(mut)	§1.1 paired- mutation meta-tests	helix_code/tests/stresschaos/ stresschaos_meta_test.go; Makefile stress-chaos-meta (:121)	PRESENT
(regr)	regression- guard	helix_code/tests/regression/	PRESENT (dir exists)

HelixCode-side coverage GAPS (must be filled or explicitly delegated-with-evidence): ddos (6), scaling (7), ux (13) have NO HelixCode-local harness — they exist only as HelixQA banks/challenge scripts. ui (12) is thin HelixCode-side. Per CONST-050(B) a delegated slot is acceptable only if the HelixQA/challenge asset truly exercises HelixCode and captures evidence; otherwise it is a §11.4 coverage bluff.

3.3 What the NEW features each need (per test type)

For each new feature (model-access, helixagent-exposure, cli-bridge, parallel-agents, dynamic-flows) the four-layer floor (§11.4.4(b)) + every applicable type:

- **model-access** (LLMsVerifier-sourced model/provider access): unit (selection/filtering); integration (real verifier data per CONST-037, real provider HTTP); e2e (/api/v1/llm/generate real output); security (key handling, no leak CONST-042); ddos/scaling (rate-limit + concurrent generate); stress/chaos (provider timeout/kill injection); performance/benchmarking (tokens/latency, ties §11.4.141); Challenge + HelixQA bank `helixcode-providers-verifier.yaml` (exists). New banks for new providers.
- **helixagent-exposure** (HelixCode capabilities exposed to/through HelixAgent): integration (cross-module call `HelixCode[?]helix_agent_cli/event_bus`); e2e (full agent-driven flow); contract tests on the shared Scheduler/Dispatcher interfaces; chaos (agent crash mid-task → consensus recovery); autonomous-QA session driving HelixAgent against HelixCode.
- **cli-bridge** (drive external CLI agents — aider/claude/cline/...): integration (spawn real CLI per `cli/<agent>/`); e2e (round-trip prompt→edit→commit); security (sandboxed shell — `tests/integration/background_shell_test.go` pattern); ui/ux (terminal interaction via HelixQA); stress (N concurrent CLI instances); Challenge banks `cli-agents-comprehensive.yaml`, `cli-agent-e2e-flow.yaml` (exist in HelixQA).
- **parallel-agents** (coordinated multi-agent default): unit (scheduler, conflict resolver); integration (subagent spawn in-process+subprocess+SSH); e2e (multi-agent workflow completes); **stress** ($N \geq 10$ parallel per §11.4.85 — extend `agent_stress_test.go/worker_pool_stress_test.go`); **chaos** (kill agent/worker mid-flight, partition — extend `worker_pool_chaos_test.go/consensus_stress_chaos_test.go`); scaling (horizontal worker scale-out); performance (throughput vs parallelism); §1.1 mutation (planted deadlock/leak detected — `stresschaos_meta_test.go` pattern).
- **dynamic-flows** (new flow_engine/dag/pipeline/mesh submodules): EACH new submodule needs its OWN full matrix (unit→Challenge) per CONST-050(B), decoupled + standalone-testable (CONST-051); HelixCode + HelixAgent each need integration+e2e adapter tests; flow definitions become HelixQA banks; chaos = node failure/partial-DAG-failure injection; stress = wide fan-out + deep nesting; ux = human-in-loop node flows.

3.4 Standing QA tasks (do not execute now)

1. `git fetch && pull` HelixQA + challenges + agentic/concurrency/planning submodules to latest; bump `.gitmodules` pointers (§11.4.71).
 2. Re-run HelixQA `docs/test-coverage.md` round to confirm 15 slots still FILLED post-pull (CONST-055).
 3. Add HelixCode-local ddos/scaling/ux harnesses OR document delegation with captured HelixQA evidence per slot.
 4. Author HelixQA banks + Challenges for each new submodule and each new feature before any close-out (§11.4.135 regression guard per fix).
-

Executive summary (12 lines)

1. **Parallel-agent capability EXISTS, twice:** HelixCode has a self-contained stack (agent/coordinator.go, agent/subagent/in-process+subprocess+worktree spawner, SSH worker/pool with consensus, DAG workflow/, autonomy+planmode).
2. HelixAgent independently has graph-flow (agentic/workflow.go WorkflowGraph), agents/swarm, ensemble, planning/{tree_of_thoughts,mcts,hiplan}, concurrency/, clis/event_bus.
3. **Gap:** the two are duplicated, not shared; HelixCode does NOT import own-org agentic/concurrency/planning (only its own internal/agent/subagent); no universal “every operation is a parallel-dispatchable agent unit” substrate; no swarm/ensemble parity in HelixCode.
4. Own-org reuse candidates already cloned: submodules/{agentic, concurrency, planning, llm_orchestrator, debate_orchestrator} — extend-don’t-reimplement (§11.4.74).
5. **Proposed NEW decoupled submodules (HelixDevelopment, public via gh/ghlab):** flow_engine (dev.helix.flow), dag_orchestrator (dev.helix.dag), pipeline_runtime (dev.helix.pipeline), agent_mesh/swarm_kit (dev.helix.mesh), optional flow_dsl (dev.helix.flowdsl).
6. **Reuse-not-new:** planning_search ← existing vasic-digital/Planning; flow_engine should EXTEND vasic-digital/Agentic graph engine where ≥80% matches.
7. **Wiring:** shared Node/Scheduler/Dispatcher/Resolver Go interfaces; HelixCode go.mod require+replace; register in coordinator.go+workflow/executor.go; HelixAgent keeps agentic as thin adapter; each submodule ships helix-deps.yaml (CONST-054, no nested own-org chains).
8. **Research topics (12, not run now):** LangGraph, Temporal/saga, Airflow/Dagster/Prefect DAG, behavior-trees vs FSM, actor/supervision, MCTS/ToT/GoT, Go structured concurrency, workflow DSL, agent-voting consensus, backpressure, AutoGen/CrewAI/Swarm, gh/ghlab multi-upstream repo creation.
9. **Testing — present:** HelixCode strong on unit/integration/e2e/security/chaos/stress/performance/benchmarking + §1.1 mutation meta-tests + regression dir; HelixQA ledger declares all 15 slots FILLED.
10. **Test-type GAPS (HelixCode-local):** ddos (6), scaling (7), ux (13) have NO HelixCode-local harness (HelixQA/challenge-only); ui (12) thin — delegated slots risk a §11.4 coverage bluff unless the delegated asset truly exercises HelixCode with captured evidence.
11. **HelixQA is STALE** (README round 219 / 2026-05-19) — must be fetched+pulled to latest and pointer-bumped (§11.4.71) before relied on; CONST-055 post-pull re-verify required.
12. **New features** (model-access, helixagent-exposure, cli-bridge, parallel-agents, dynamic-flows) each need the full per-type matrix + four-layer floor; every new flow submodule needs its OWN standalone matrix + HelixQA banks + Challenges + §11.4.135 regression guards before close-out.